

Programming Assignment 1
CS6903: Network Security
2025-26 - Semester II

Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad

Due Date: 14th February 2026, 11:55pm [*No extension]

This programming assignment acquaints you with the working principles of Cryptography and secure communication. You need to establish a communication channel between two processes (Client/Server) to achieve different functionalities as described in each task.

Please note that your submission will be evaluated on a Linux machine. Therefore, ensure that your programming platform is compatible with the Linux environment. Submissions that fail to execute on Linux will receive zero marks.

Allowed Libraries:

- AES encryption libraries
- DES encryption libraries
- Standard SHA-256 HASH library

Not Allowed:

- High level socket programming libraries or frameworks
- High-level networking APIs

Task 1: Basic File Management Service [15 Points]

In this task, you need to implement a remote file management service using client-server architecture.

Requirement specifications for this task are as follows:

1. The client reads one command at a time from the terminal.
2. All commands are UPPERCASE ASCII strings.
3. Each command is sent as a single request to the server.
4. The server processes commands sequentially per client connection.
5. Responses are plain-text strings.

Supported Commands

- **LIST** Server returns the list of files in a predefined directory.
- **INFO <filename>** Server returns metadata of the specified file such as last modified time, creation time, and permissions.
- **GETSIZE <filename>** Server returns the size of the specified file in bytes.
- **QUIT** Client terminates the connection.

Logging Requirement

The server must maintain a log file at server side named **server.log**. For every client request and response, the server must log:

- Timestamp
- Command received
- Response sent

Example log file:

```
[2026-02-03 10:15:22] REQUEST: LIST
[2026-02-03 10:15:22] RESPONSE: file1.txt, file2.txt
```

Example Input and Output at client side

```
[INPUT]  LIST
[OUTPUT] RESPONSE: file1.txt file2.txt

[OUTPUT] RESPONSE: 2048 bytes

[INPUT]  QUIT
```

Task 2: Authentication with Multiple Clients and Session Key Establishment [25 points]

In this task, you will extend Task 1 by adding authentication, Multiple Clients support, and secure key establishment.

Multiple Clients Requirement

- The server must support multiple clients simultaneously.
- Each client connection must be handled using a separate thread/process.

Authentication Mechanism

- Server maintains a credentials file containing a list of usernames and corresponding shared_secret:

```
username: shared_secret
```

Authentication Protocol

1. Client sends username.
2. Server generates and sends a random nonce.
3. Client computes: $HASH(nonce || shared_secret)$
4. Server verifies the response.

Note: The client should be permitted to perform file management operations only after successful authentication.

Account Lockout Policy

If a user enters incorrect authentication credentials **three consecutive times**, the server must:

- Block that username for the current session.
- Terminate the connection.

Session Key Establishment

After successful authentication, the client and server must establish a shared secret (session key) using the Diffie-Hellman key exchange protocol.

- The client reads values of p and g from environment variables **P** and **G**.
- Both client and server generate private secret numbers.
- Using the Diffie-Hellman protocol, they compute a shared secret value.
- This shared secret will be used as the **session key** for all further secure communication and integrity verification.

The derived session key must be identical at both the client and the server after the exchange.

Task 3: File Download with Integrity [30 points]

In this task, you are required to implement a secure file download mechanism from the server while ensuring the file's integrity. This task builds upon Task 2, where you will extend the existing implementation to include the file download functionality.

Use of Session Key

To securely download a file from the server while preserving its integrity, you must use the session key generated through the Diffie-Hellman key exchange. No other keys should be used for verifying the file's integrity.

File Transfer Command

Introduce a command **GET <filename>**. When the server receives this command, it should begin sending the requested file to the client. If the requested file does not exist on the server, the server should send a **"FILE NOT AVAILABLE"** message to the client.

Transfer Mechanism

- The file must be divided into fixed-size chunks of **1024 bytes**.
- The last chunk may contain fewer bytes if the file size is not a multiple of **1024**.
- Send each chunk in the following format:

SEQ_NO | DATA | MAC

Where MAC is being calculate as follows:

MAC = HASH(DATA || SEQ_NO || SESSION_KEY)

Server Responsibilities

Server must:

- Verify MAC of every chunk using the session key derived in Task 2
- Reject modified, missing, or out-of-order packets
- Reconstruct file only after successful verification

Task 4: Encryption of Communication [30 points]

So far, you have authenticated the user, successfully established a session key, and transferred files while ensuring their integrity. Now, you need to secure all subsequent communication using encryption.

Protocol Flow

- Authentication (from Task 2) remains in plaintext.
- After successful authentication, Diffie-Hellman key exchange is performed.
- After key exchange, all further communication must be encrypted.

Requirements

- Use AES for encrypting the communication.
- Derive a 128-bit AES key from the session key using any suitable method.
- Use this derived AES key to encrypt all commands, server responses, and file data.
- Ensure that the integrity protection implemented in Task 3 continues to be applied.

Chunk-Level Encryption Requirement

- For file transfer operations, encryption must be performed on a per-chunk basis.
- The entire file must not be encrypted as a single unit.
- Each transmitted packet must follow the format:

SEQ_NO | ENC(DATA) | MAC

where:

```
ENC(DATA) = AES encrypted chunk data  
MAC = HASH(ENC(DATA) || SEQ_NO || SESSION_KEY)
```

The server must first verify the MAC and then decrypt each chunk to obtain the original data.

Example Input and Output at client side

```
[INPUT]  LIST  
[OUTPUT] RESPONSE: file1.txt file2.txt
```

```
[INPUT]  GET sample.txt
```

```
[OUTPUT] [Cipher]: <ciphertext>  
[OUTPUT] [Plain]: <plaintext>
```

```
[OUTPUT] [Cipher]: <ciphertext>  
[OUTPUT] [Plain]: <plaintext>
```

```
[INPUT]  QUIT
```

Output Format

```
[Cipher]: <ciphertext>  
[Plain]: <plaintext>
```

Deliverable:

Source Code:

- Submit the source code for **Task 1, Task 2, Task 3, and Task 4.**
- Document your code properly with comments and meaningful variable names.
- Place each task's source code in a separate folder named with the Task number.

README.txt:

Create a README.txt file containing the following:

- Clear instructions on how to compile and run the code for each task.
- Steps required for execution and any necessary environment setup.
- Details of any external libraries used.
- Group member name and roll number.
- **Contribution of the group members, specifying what has been done by whom.**
- List all references used while completing the assignment, including any discussions or collaborations with other groups.
- You should not copy code directly from others.

Assignment Report (Mandatory):

Students must submit a detailed assignment report containing:

- Summary and design overview of each task.
- Explanation of the implementation approach for Task 1, Task 2, Task 3, and Task 4.
- Screenshots or sample outputs demonstrating correct execution.
- For **Task 2, Task 3, and Task 4**, a detailed network traffic analysis using Wireshark:
 - Explanation of observed network packets.
 - Screenshots of relevant Wireshark captures.
 - Observations on plaintext communication (where applicable).
 - Demonstration of encrypted communication for Task 4 using Wireshark screenshots.
- Challenges faced during implementation and how they were resolved.

Submission Format:

- Submit a **tar-ball/zip** file of your assignment named as your **roll number** (e.g., 12345678.tar.gz).
- The zip file should contain the Source code, the README.txt file, and the Assignment Report (PDF).
- Any one member from the group can submit the assignment.
- Upload your submission to the **IITH CSE Moodle platform**. Submissions via email or any other medium are strictly prohibited.

All the best. Looking forward to the submissions !!.....PS: Start Early.

ANTI-PLAGIARISM STATEMENT <Include it in your report>

We certify that this assignment/report is our own work, based on our personal study and/or research and that we have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication. We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, We understand my responsibility to report honor violations by other students if we become aware of it.

Names:

Date:

Signature: <keep your initials here>